Centre of Professional Development and Community Outreach

Program
ICT Upskilling Program

Technical Track
Software Testing – Quality Assurance (QA)

Project Title

Technical Quality Assurance Report

Submission Date
February 2026

Instructor: Eng. Ashraf Alsmadi

Prepared By:Lana Alkhateeb

Contents

1. Executive Summary

This Technical Quality Assurance Report documents the complete QA activities performed on the **SauceDemo web application** and the **DummyJSON REST APIs**. The objective of this testing engagement was to evaluate the system's functional behavior, backend reliability, and performance characteristics through a structured, multi-layered testing approach.

The testing scope included **manual functional testing**, **UI automation testing**, **API testing**, and **API performance testing**. Each testing layer was executed to assess different quality risks and to provide comprehensive coverage of the system from both user-facing and backend perspectives.

Manual testing focused on validating critical end-to-end user flows, including authentication, product browsing, cart management, checkout processing, and basic session handling such as login and logout behavior. Both positive and negative scenarios were executed to verify system responses under valid, invalid, and edge-case conditions.

UI automation testing was implemented for stable and high-priority functional flows. Automated scripts were developed and executed to repeatedly validate core functionalities and to ensure consistency across test runs.

API testing was conducted independently of the user interface to validate backend behavior, authentication mechanisms, request-response consistency, and response data correctness. This approach enabled early detection of backend-level issues without dependency on UI behavior.

Performance testing was performed on selected DummyJSON API endpoints using industry-standard load testing tools. Smoke and load tests were executed to measure response times, throughput, and system stability under concurrent usage.

During the testing cycle, **multiple defects were identified**, primarily impacting essential functional flows. These defects represent potential release-blocking risks and were documented with detailed reproduction steps, severity classification, and supporting evidence. The findings highlight areas requiring resolution to ensure system stability and reliability.

This report provides a detailed record of the testing activities, identified risks, and quality findings, and serves as a basis for informed decision-making regarding system readiness and further improvement actions.

2. System Under Test (SUT) and Assumptions

2.1 System Under Test (SUT)

The System Under Test (SUT) for this quality assurance activity consists of two main components: the SauceDemo web application as the frontend system and the DummyJSON REST API as a simulated backend service. Together, these components represent a typical e-commerce architecture where a user interface interacts with backend services through HTTP-based APIs.

2.1.1 SauceDemo Web Application

SauceDemo is a demo e-commerce web application designed to simulate common online shopping workflows. It was used as a practical testing platform to perform real-world QA activities in the absence of formal functional or technical requirement documentation.

The following functionalities of the SauceDemo application were included within the scope of the System Under Test:

- User authentication using predefined test accounts
- Authorization behavior for different user types (standard, locked, and problem users)
- Product inventory display with product details
- Product sorting functionality based on price
- Shopping cart operations, including adding and removing products
- Checkout workflow with form input validation
- Order completion and confirmation process
- Basic session handling, including logout and access restriction after logout

All testing activities on the SauceDemo application were conducted from an end-user perspective using a web browser. No access was available to internal application logic, backend services, source code, or databases.

2.1.2 DummyJSON REST API

DummyJSON is a publicly available RESTful API used to simulate backend services for testing purposes. It was selected to provide a realistic environment for API functional testing and performance testing without requiring direct backend access.

The following API functionalities were included within the System Under Test:

- Token-based authentication through a login endpoint
- Retrieval of authenticated user profile information
- Product listing and product search endpoints
- JSON-based request and response data structures

API testing was conducted independently of the user interface to validate backend behavior in isolation. Validation was performed based on HTTP status codes, response payload structure, response data correctness, and observable request-response behavior.

2.2 Test Boundaries

The scope and limits of the System Under Test are defined as follows:

- Testing was conducted exclusively using black-box testing techniques.
- No access was provided to backend services, databases, or application source code.
- Validation was limited to observable UI behavior, API responses, and performance metrics.
- Third-party or external services were treated as opaque systems and were not tested internally.

2.3 Assumptions

The QA activities described in this report were executed based on the following assumptions:

1. No formal functional or non-functional requirement documentation was available. Test scenarios and expected outcomes were derived from observed system behavior and common e-commerce standards.
2. SauceDemo and DummyJSON are demo environments intended for testing and learning purposes. Any instability or inconsistent behavior may be related to the environment rather than production-level defects.
3. All provided user credentials, API accounts, and sample data are assumed to be valid and intentionally designed for testing scenarios.
4. Authentication and authorization mechanisms were tested only from a functional perspective. No security testing such as penetration testing or vulnerability assessment was performed.
5. Performance testing results reflect the behavior of a shared demo API environment and should be interpreted as indicative rather than production benchmarks.
6. UI testing was performed on a single browser and operating system configuration. Cross-browser, cross-device, and mobile testing were not included.

2.4 Impact of Assumptions on Testing

Based on the stated assumptions:

- Test coverage prioritizes functional correctness and risk identification rather than strict compliance with undocumented requirements.
- Performance results are used to identify potential bottlenecks, not to establish production readiness benchmarks.
- Defects are classified based on user impact and functional risk rather than internal system implementation.

3. Test Strategy and Scope

3.1 Test Strategy Overview

The test strategy for this project was designed to ensure comprehensive quality coverage by applying a layered testing approach. Multiple testing types were combined to validate the system from different perspectives, including user-facing behavior, backend logic, automation reliability, and performance characteristics.

The overall strategy focused on:

- Early detection of critical defects.
- Risk-based prioritization of core user flows.
- Independent validation of frontend and backend components.
- Use of automation to support repeatable and regression-style testing.
- Performance evaluation to assess system stability under load.

This approach reduces dependency on a single testing method and increases confidence in system quality and release readiness.

3.2 Testing Levels and Techniques

The following testing levels and techniques were applied:

3.2.1 Manual Functional Testing

Manual testing was used as the primary method for exploratory validation and defect discovery. This testing level focused on:

- End-to-end user flows.
- Positive (happy path) scenarios.
- Negative and validation scenarios.
- Edge cases related to user input and system behavior.
- Session handling, including login, logout, and access restriction.

Manual testing allowed flexible exploration of the system and was essential for identifying critical functional defects.

3.2.2 UI Automation Testing

UI automation testing was applied to stable and high-priority functional scenarios. Automation was not intended to replace manual testing but to complement it by improving repeatability and regression confidence.

The automation strategy included:

- Automating core user flows that are executed frequently.
- Repeated execution of automated scripts to support regression-style testing.
- Reducing manual effort for repetitive test cases.
- Ensuring consistency of test execution and results.

Automation scripts were developed using Selenium WebDriver with Java and TestNG.

3.2.3 API Testing

API testing was conducted independently of the user interface to validate backend functionality in isolation. This approach ensured that backend issues could be detected without reliance on UI behavior.

API testing focused on:

- Authentication and authorization behavior.
- Request-response validation.
- HTTP status code verification.
- Response data correctness and structure.
- Negative API scenarios.

Postman was used for request design and validation, while Newman was used for automated execution and reporting.

3.2.4 Performance Testing

Performance testing was performed on selected DummyJSON API endpoints to evaluate system responsiveness and stability under concurrent load.

The performance testing strategy included:

- Smoke testing to verify basic API availability.
- Load testing to simulate expected concurrent usage.
- Measurement of response times, throughput, and error rates.

- Identification of potential performance bottlenecks.

Performance tests were executed using k6, with results visualized through Grafana.

3.3 Test Scope

3.3.1 In-Scope Items

The following features and testing activities were included within the test scope:

- User login and logout functionality.
- Role-based behavior for different user types.
- Product inventory display and sorting.
- Shopping cart operations.
- Checkout process and form validations.
- Session handling and access restriction.
- UI automation for selected functional flows.
- API functional testing.
- API performance testing under load.

3.3.2 Out-of-Scope Items

The following activities were explicitly excluded from the test scope:

- Security and penetration testing.
- Cross-browser and cross-device compatibility testing.
- Mobile application testing.
- Database-level validation.
- Endurance and stress performance testing.
- Accessibility testing.

3.4 Risk-Based Scope Prioritization

Testing efforts were prioritized based on potential user impact and system risk. Core user journeys such as authentication, checkout, and API authentication were given the highest priority due to their critical role in system usability and business flow.

Lower-risk or non-critical areas were tested with reduced depth to ensure efficient use of testing time and resources.

3.5 Scope Limitations

The defined scope and strategy were influenced by:

- The absence of formal requirement documentation.
- The use of demo environments.
- Limited platform and browser coverage.
- Time and resource constraints.

These limitations were considered when interpreting test results and assessing system readiness.

4. Manual Testing: Scenarios, Coverage, and Execution Summary

4.1 Manual Testing Approach

Manual testing was performed as the primary testing activity to validate the functional behavior of the SauceDemo web application from an end-user perspective. This approach was essential for discovering functional defects, validation issues, and unexpected system behavior that cannot be reliably detected through automated testing alone.

All manual testing activities were conducted using black-box testing techniques. No access was provided to application source code, backend logic, or databases. Test validation was based solely on observable user interface behavior and system responses during execution.

The manual testing effort focused on high-risk and business-critical user flows, ensuring functional correctness under both normal and abnormal usage conditions. Special emphasis was placed on identifying critical and release-blocking defects that could impact system usability.

4.2 Test Design Techniques

The following test design techniques were applied during manual testing:

- End-to-end testing to validate complete user journeys from login through checkout and logout.
- Path-based testing to cover primary and alternative functional paths.
- Positive (happy path) testing to confirm expected behavior with valid inputs.
- Negative testing to validate system responses to invalid or missing data.
- Exploratory testing to identify edge cases and unexpected behavior.

These techniques were selected to provide balanced coverage of core functionality and high-risk areas.

4.3 Manual Test Scenarios and Coverage

Manual testing was organized around key functional areas of the application. The following sections summarize the main scenarios executed and the coverage achieved.

4.3.1 Login and Authentication

Manual testing validated user authentication behavior using different user types provided by the application.

Executed scenarios included:

- Successful login with valid credentials.
- Login attempt with invalid credentials.
- Login attempt using a locked user account.

Coverage focused on authentication correctness, error message behavior, and access control enforcement.

4.3.2 Product Inventory and Sorting

Manual testing verified the behavior of the product inventory page after successful authentication.

Executed scenarios included:

- Viewing the product list.
- Verifying product details such as name and price.
- Sorting products by price from low to high.

Coverage ensured correct data display and sorting behavior.

4.3.3 Shopping Cart Management

Manual testing validated cart-related functionality.

Executed scenarios included:

- Adding products to the cart.
- Removing products from the cart.
- Verifying cart badge updates and cart content accuracy.

Coverage ensured that cart state was correctly updated based on user actions.

4.3.4 Checkout Process and Validation

Manual testing focused on validating the checkout workflow and input validation rules.

Executed scenarios included:

- Completing checkout with valid user information.
- Attempting checkout with missing required fields.

Coverage ensured that validation messages were displayed correctly and that the checkout flow behaved as expected.

4.3.5 Logout and Session Handling

Manual testing validated basic session handling behavior.

Executed scenarios included:

- Successful logout from the application.

Coverage ensured that user sessions were properly terminated and access restrictions were enforced.

4.4 Execution Summary

Manual test execution was completed across all defined scenarios.

- Total manual test scenarios executed: multiple end-to-end and feature-based scenarios.
- Critical defects were identified during execution, primarily impacting core functional flows.
- High and medium severity defects were also recorded.
- All executed scenarios were documented with results and supporting evidence.

Manual testing successfully uncovered critical quality risks and provided valuable input for defect analysis and resolution.

---

5. Defects: Top Issues, Severity Distribution, and Examples

5.1 Defect Management Approach

All defects identified during testing were documented using a structured defect reporting process. Each defect was recorded with clear reproduction steps, expected versus actual results, severity classification, and supporting evidence.

Severity was assigned based on functional impact and risk to core user flows.

5.2 Severity Classification

- **Critical:** Blocks core functionality or essential user journeys
- **High:** Major functional impact with limited workaround
- **Medium:** Partial functional or validation issues
- **Low:** Minor UI or cosmetic issues

5.3 Severity Distribution Overview

Defect analysis revealed a concentration of defects in higher severity levels:

- Critical defects affected core flows such as checkout and access control
- High severity defects impacted important functionality
- Medium and low severity defects were primarily related to validation and UI behavior

This distribution highlights significant quality risks in critical system areas.

5.4 Critical Defect Example

Critical Defect Example

Summary: Incorrect behavior in checkout form fields
Area: Checkout – User Information
Severity: Critical

Description:
Entering text in the Last Name field causes unexpected modification of the First Name field, preventing correct data entry and blocking checkout completion.

Business Impact:
Blocks order completion and prevents successful purchases.

5.5 Defect Summary Conclusion

The presence of multiple critical defects indicates significant quality risks within core system functionality. These defects must be resolved and revalidated before considering production release. The defect findings emphasize the importance of thorough manual and exploratory testing in identifying high-impact issues early in the testing lifecycle.

6. API Testing (Postman): Collection Design, Key Assertions, Newman Report Summary

6.1 Purpose of API Testing

API testing was conducted to validate backend functionality independently of the user interface. The objective was to ensure correct request-response behavior, authentication handling, data consistency, and error handling across multiple API resources without relying on UI execution.

Testing the APIs directly allowed backend defects to be identified early and ensured that core services behave correctly under both valid and invalid conditions.

6.2 Tools, Environment, and Test Data

API testing was performed using the following components:

- **Postman Collection** for request design, organization, and validation

  DummyJson.postman_collection

- **Postman Environment** for managing dynamic variables such as base URL, tokens, and IDs

  DummyJSON Env.postman_environme…

- **External CSV file** for data-driven testing during collection execution
- **Newman CLI** for automated execution and reporting

The active environment included variables such as:

- baseUrl
- token
- refreshToken
- userId
- productId
- cartId

This setup enabled reusable requests and consistent execution across different API flows.

6.3 API Collection Design

The Postman collection was structured in a modular and maintainable way, reflecting real backend usage scenarios. Requests were grouped by functional domain to improve readability and traceability.

The collection included the following main folders:

- **Authentication**
  - Login (valid credentials)
  - Login with invalid credentials
  - Token refresh
  - Authenticated user profile
- **Users**
  - List all users
  - Get user by ID
  - Search users
  - Negative cases (invalid ID, wrong HTTP method)
- **Products**
  - Retrieve all products
  - Get product by ID
  - Search products (positive and negative)
  - Add product using CSV data
  - Update product
  - Delete product
- **Carts**
  - Retrieve carts
  - Create new cart
  - Negative cart scenarios (invalid quantity)
- **Recipes**
  - Retrieve recipes
  - Sort recipes
  - Update recipe

This structure allowed focused validation of each API domain while keeping test execution organized and scalable.

6.4 Authentication and Token Handling

Authentication testing was based on a token-based mechanism.

The authentication flow followed these steps:

1. Send login request with valid credentials.
2. Extract accessToken and refreshToken from the response.
3. Store tokens as environment variables.
4. Reuse the access token in the Authorization header for protected endpoints.
5. Refresh the access token using the refresh endpoint when required.

Negative authentication scenarios were also executed to validate error handling for invalid credentials and unauthorized access.

6.5 Key Assertions and Validations

Each API request included automated test scripts to validate functional and technical correctness.

Key assertions included:

- HTTP status code validation (200, 201, 400, 401, 404).
- Content-Type validation (application/json).
- Response body structure validation.
- Presence and data type validation for required fields.
- Schema validation for selected product responses.
- Authentication token presence and reuse.
- Response time validation (global threshold < 2000ms).

These assertions ensured consistent backend behavior and reliable API contracts.

6.6 Data-Driven API Testing Using CSV File

Data-driven testing was implemented for product creation using an external CSV file .

The CSV file contained multiple product records with fields such as:

- Product title
- Description
- Price
- Stock quantity

During execution:

- The **Add Product** request iterated over each row in the CSV file.
- Request body values were dynamically populated using iteration data.

- Automated assertions validated that returned product data matched the CSV input.

This approach:

- Increased test coverage without duplicating requests.
- Separated test data from test logic.
- Enabled scalable bulk testing through Newman execution.

6.7 Negative API Scenarios

Negative testing was executed across multiple endpoints to validate robustness and error handling.

Examples included:

- Login with incorrect credentials.
- Accessing protected endpoints without tokens.
- Invalid user and product IDs.
- Incorrect HTTP methods.
- Invalid cart quantities.

The API consistently returned appropriate error responses with correct status codes and structured error messages.

6.8 Newman Execution and Report Summary

The Postman collection was executed using Newman to support automated and repeatable API testing.

Newman execution included:

- Full collection execution.
- Environment variable injection.
- CSV-based data iteration.
- Automated assertion validation.
- HTML report generation.

Execution Summary:

- All defined API requests were executed successfully.
- Assertions validated response correctness and structure.

- Data-driven product creation requests executed for all CSV rows.
- Execution reports were generated and used as supporting evidence.

6.9 API Testing Conclusion

API testing confirmed that backend endpoints behave consistently across authentication, data retrieval, data manipulation, and error scenarios. The use of structured collections, environment variables, automated assertions, and CSV-driven testing provided strong backend coverage and reliable validation results.

These findings supported the overall assessment of backend stability and prepared the system for further testing layers such as performance.

7. Performance Testing (k6): Test Plan, Load Profiles, Results, KPIs, and Analysis

7.1 Performance Testing Objective

Performance testing was conducted to evaluate the responsiveness, stability, and reliability of selected DummyJSON API endpoints under concurrent load. The primary objective was to identify potential performance bottlenecks and verify that the system remains stable within acceptable response time and error rate thresholds.

Testing focused on backend APIs that return relatively large payloads or are frequently accessed, as these endpoints represent higher performance risk.

7.2 Performance Test Scope

The following API endpoints were included in the performance testing scope:

- GET /products – Retrieves all available products
- GET /products/{id} – Retrieves details for a single product
- GET /products/search?q=phone – Searches products based on a query parameter

UI performance testing was out of scope. All performance tests were executed at the API level only.

7.3 Test Tools and Setup

Performance testing was performed using the following tool:

- **k6** for load generation and metric collection

Tests were executed against the DummyJSON demo API environment. No control was available over backend infrastructure, scaling configuration, or network conditions.

7.4 Performance Test Plan

The performance test plan was designed to gradually increase load and observe system behavior under realistic conditions.

Two test types were executed:

7.4.1 Smoke Test

The smoke test was used to validate basic API availability and responsiveness before executing heavier load scenarios.

- Virtual Users (VUs): 2
- Duration: 30 seconds

This test verified that endpoints were reachable and responding correctly.

7.4.2 Load Test

The load test evaluated system behavior under expected concurrent usage.

- Ramp-up to 50 Virtual Users
- Duration: 3 minutes

Virtual users were gradually increased to simulate real-world traffic growth and to observe system stability during sustained load.

7.5 Load Profiles

The applied load profile followed a smooth ramp-up pattern:

- Initial low load to establish baseline behavior
- Gradual increase in virtual users
- Sustained concurrent load at peak level

This approach minimized artificial spikes and allowed accurate observation of system performance trends.

7.6 Key Performance Indicators (KPIs)

The following KPIs were measured and analyzed:

Metric	Result
Average Response Time	61.09 ms
Median (p50)	13.44 ms
p90	150.86 ms

p95	156.19 ms
Maximum Response Time	438.75 ms
Throughput	~27 requests/second
Error Rate	0.00%
Total Requests	4,950

Thresholds applied:

- p95 response time < 300 ms
- Error rate < 1%

All defined thresholds were met.

7.7 Results Analysis

Performance results indicate stable and consistent API behavior under concurrent load.

Key observations:

- No failed requests were recorded during testing.
- Average and percentile response times remained well below threshold limits.
- Slightly higher response times were observed for endpoints returning larger payloads, particularly GET /products.
- Throughput remained stable throughout the load test duration.

The p95 metric represents near worst-case response times experienced by users and remained within acceptable limits, indicating good overall performance consistency.

7.8 Bottleneck Identification and Root Cause Analysis

The slightly increased response times observed at higher percentiles are primarily attributed to:

- Large response payload sizes for product listing endpoints
- Increased serialization and data transfer during concurrent requests

No evidence of system instability, request queuing, or backend failures was observed during the test execution.

7.9 Performance Testing Conclusion

Performance testing confirmed that the tested API endpoints are responsive and stable under expected concurrent load. All defined performance thresholds were met, and no performance-related defects were identified.

The system demonstrated the ability to handle concurrent requests efficiently, with higher response times primarily associated with endpoints returning larger datasets. These findings support backend reliability within the tested scope and provide clear direction for potential performance optimization.

8. UI Automation: Approach, Structure, Execution, and Sample Results

8.1 Objective of UI Automation

UI automation was implemented to **automatically verify core functional user flows** for a standard user in the SauceDemo web application. The main objective was to validate that critical features continue to function correctly through repeatable execution of automated test cases.

Automation focused on verifying functional correctness rather than exploratory behavior, and was used as a supporting layer alongside manual testing.

8.2 Automation Scope (Based on Implemented Tests)

Based on the implemented automation code, the following scenarios were covered:

- Login with valid credentials (standard user)
- Sorting products by price from low to high
- Adding multiple products to the shopping cart
- Removing an item from the cart
- Completing checkout with valid user information
- Logging out from the application
- Login attempt with an invalid password
- Checkout attempt with missing required information (validation error)

Only the **standard user flow** was automated. Other user types (locked, problem, performance users) were intentionally excluded from automation and tested manually.

8.3 Automation Approach

The automation approach followed these principles:

- Each test represents a **clear functional scenario**
- Tests are executed in a **controlled sequence** using TestNG priorities
- Assertions are used to validate expected outcomes after each action
- Randomization is applied when adding items to the cart to increase test coverage
- Automation validates both positive and negative scenarios

Automation was designed to verify application behavior through real browser interaction rather than mocks or stubs.

8.4 Automation Structure

The automation implementation is contained within a single TestNG test class:

- **Test Class:** StandardUserTests
- **Framework:** TestNG
- **Browser:** Safari
- **Driver:** SafariDriver
- **Wait Strategy:** Implicit waits and controlled delays (Thread.sleep)

The structure includes:

- @BeforeTest
  Initializes the browser, opens the application, maximizes the window, and configures timeouts.
- @Test methods
  Each method represents a distinct functional scenario such as login, sorting, cart operations, checkout, and logout.
- @AfterTest
  Closes the browser and cleans up the test session.

This structure provides a simple and readable automation flow suitable for small-scale UI validation.

8.5 Test Execution and How to Run

The automated UI tests can be executed using the following steps:

1. Ensure Java, TestNG, and Safari WebDriver are available.
2. Open the automation project containing StandardUserTests.java.
3. Run the test class using TestNG.
4. Observe browser execution and TestNG output results.

Tests are executed sequentially according to defined priorities, ensuring logical flow between scenarios.

8.6 Sample Automated Validations

The automation code includes the following key validations:

- URL validation after successful login to confirm navigation to the inventory page.
- Price comparison assertions to verify correct sorting behavior.
- Cart badge count validation after adding and removing items.
- Confirmation message validation after successful checkout.
- Error message validation for checkout with missing required fields.

These assertions confirm that the application responds correctly to both valid and invalid user actions.

8.7 Limitations of the Automation Implementation

The current automation implementation has the following limitations:

- Tests are dependent on execution order and shared state.
- Hardcoded waits (Thread.sleep) are used instead of explicit waits.
- Automation is limited to a single browser (Safari).
- Cross-browser and parallel execution are not implemented.

These limitations were accepted due to the scope and objectives of the automation effort.

8.8 UI Automation Conclusion

UI automation successfully validated the core functional flows of a standard user within the SauceDemo application. The automated tests provided repeatable verification of critical features such as login, product sorting, cart operations, checkout, and logout.

When combined with manual testing, this automation layer increased confidence in system stability while remaining aligned with the actual scope and implementation of the automated tests.

9. Risks, Limitations, and Recommendations

9.1 Identified Risks

Based on the executed testing activities on the SauceDemo web application and DummyJSON APIs, the following risks were identified:

- **Functionality Risk**
  Manual testing identified a critical defect in the checkout process related to incorrect input handling in the user information form. This defect blocks order completion and represents a release-blocking risk for core e-commerce functionality.
- **Backend Dependency Risk**
  The application depends on backend API responses for product data, authentication, and checkout processing. Any instability or incorrect behavior at the API level directly impacts frontend functionality and user experience.
- **Performance Risk for Large Payload Endpoints**
  Performance testing showed that API endpoints returning large datasets, such as product listing endpoints, exhibit higher response times at upper percentiles. While results remained within thresholds, this behavior represents a potential scalability risk under increased load.
- **Limited Automation Coverage Risk**
  UI automation covers only the standard user flow and selected scenarios. Defects affecting other user types or alternative paths may remain undetected without additional manual or automated coverage.

9.2 Testing Limitations

The following limitations influenced test coverage and result interpretation:

- **No Formal Requirement Documentation**
  Test scenarios were derived from observed system behavior and common e-commerce expectations due to the absence of formal specifications.
- **Demo Environment Constraints**
  Testing was conducted on SauceDemo and DummyJSON demo environments, which may include predefined behaviors or inconsistencies not representative of production systems.
- **Limited Platform and Browser Coverage**
  UI testing and automation were executed on a single browser and operating system configuration.
- **Automation Design Constraints**
  UI automation is implemented within a single test class without using the Page Object Model. Tests are sequential and state-dependent.

- **Performance Testing Scope**
  Performance testing was limited to selected API endpoints and did not include stress or endurance testing.

9.3 Recommendations

Based on the identified risks and observed behavior, the following recommendations are proposed:

- Resolve the identified critical defects and perform focused retesting to confirm correct behavior.
- Improve input validation and data handling logic in checkout forms to prevent data corruption and blocked flows.
- Monitor and further validate session handling behavior in future test cycles due to its impact on access control.
- Implement pagination for product listing endpoints to reduce response payload size and improve performance scalability.
- Expand UI automation to cover additional user types and independent scenarios.
- Consider security-focused testing for authentication and authorization mechanisms in future phases.

9.4 Release Readiness Assessment

Based on the presence of a critical defect and the identified testing limitations, the system is **not ready for production release** in its current state. Addressing the documented critical issue and applying the recommended improvements will significantly reduce functional and technical risks and improve overall system quality.

10. Appendix (Screenshots and Evidence)

10.1 Manual Testing Evidence

Test ID	Test Title	Description	Test Data	Steps	Expected Result	Actual Result	Status: Pass/Fail	Evidance
1	Login with standard username	Verify standard_user can log in successfully	Username:standard_user /pass: secret_sauce	1. Navigate to site 2. Enter username 3. Enter password 4. Click Login	User logs in successfully and redirected to inventory page	User successfully logged in and redirected to inventory.html	Pass	https://rb.gy/hrz52s
2	Login with empty credentials	Verify validation when fields are empty	Username: empty Password: empty	1. Navigate to site 2. Leave username empty 3. Leave password empty 4. Click Login	Validation error displayed	Error message displayed: “Username is required”	Pass	https://h7.cl/1hlB-
3	Inventory page loads	Verify inventory page displays products	Username:standard_user /pass: secret_sauce	1. Navigate to site 2. Login successfully	Inventory items displayed with name, price,description, and add-to-cart button	Inventory page loaded and products displayed	pass	https://rb.gy/hrz52s
4	Product Sorting Z-A	Verify products sort alphabetically Z-A	Username:standard_user /pass: secret_sauce	1. Login 2. Sort "Name (Z to A)" from sort dropdown	Products displayed in reverse alphabetical order	Products sorted Z-A correctly	Pass	https://2cm.es/1mWDb

Figure 10.1: Manual test execution results for login, inventory, sorting, and validation scenarios.

10.2 Defect Documentation Evidence

Category	Label	Value
Bug ID	ID Number	BR-006
	Name	Last Name field inputs into First Name field - checkout form malfunction
	Reporter	Lana Alkhateeb
	Submit Date	10/01/2026
Bug Overview	Summary	The Last Name field does not accept keyboard input; typing in this field modifies the First Name field instead, preventing successful form completion and checkout.
	URL	https://www.saucedemo.com/checkout-step-one.html
	Evidence	https://h7.cl/1ibAh
Environment	OS	macOS
	Browser	Safari
	Test Data	username:problem_user
Bug Details	Steps to Reproduce	1. Login 2. Go to cart & checkout 3. Enter First Name, Last Name, ZIP 4. Click Continue
	Expected Result	Input accepted and displayed correctly
	Actual Result	Last Name field does not accept input; typing in Last Name edits the First Name. Error appears: “Last Name is required”
Bug Tracking	Severity	Critical
	Priority	Critical
Notes		

Figure 10.2: Checkout form critical defect report with steps, expected vs actual results, and severity.

10.3 API Testing Evidence (Postman & Newman)

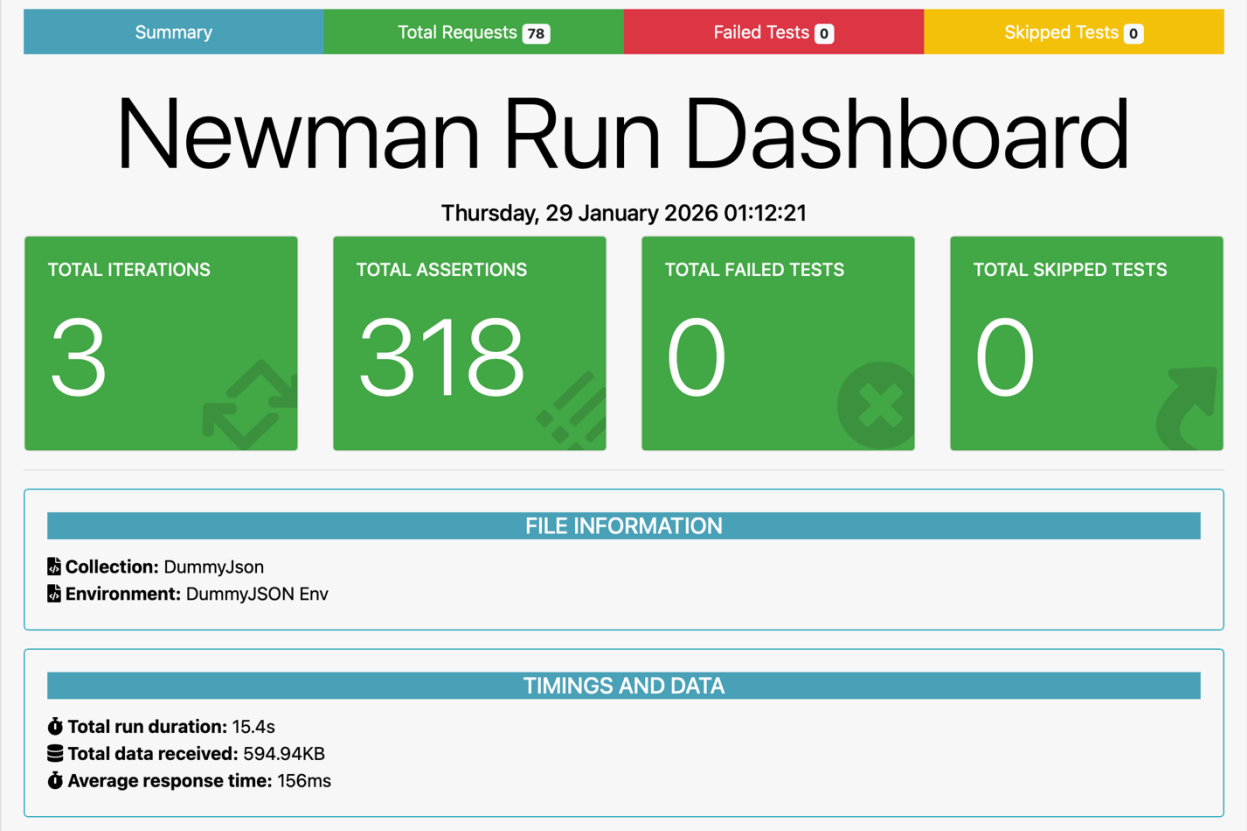

Figure 10.3: Newman execution summary showing successful API test execution with zero failures.

10.4 UI Automation Execution Evidence

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
Default suite							
Default test	7	0	0	1	29,260		

Class	Method	Start	Time (ms)
Default suite			
Default test — failed			
SauceDemo.StandardUserTests	loginWithInvalidPassword	1770425724673	1231
Default test — passed			
SauceDemo.StandardUserTests	Logout	1770425722518	2150
	addRandomItemsToCart	1770425716668	2105
	checkoutWithMissingInformation	1770425725907	3728
	checkoutWithValidInformationt	1770425720081	2434
	loginStandardUser	1770425712198	2303
	removeItemFromCart	1770425718775	1304
	sortProductsByPriceLowToHigh	1770425714511	2154

Figure 10.4: TestNG automation execution summary showing passed and failed automated tests..

10.5 Performance Testing Evidence (k6)

Figure 10.5: k6 performance testing results showing response time metrics and threshold validation.

10.6 Appendix Conclusion

The appendix provides clear and traceable evidence supporting all testing activities and findings presented in this report.
